

University Information Chatbot

A-to-Z Project Process and Technical Documentation

A complete description of how the PDF-grounded multilingual RAG system is built, executed, evaluated, and troubleshot.

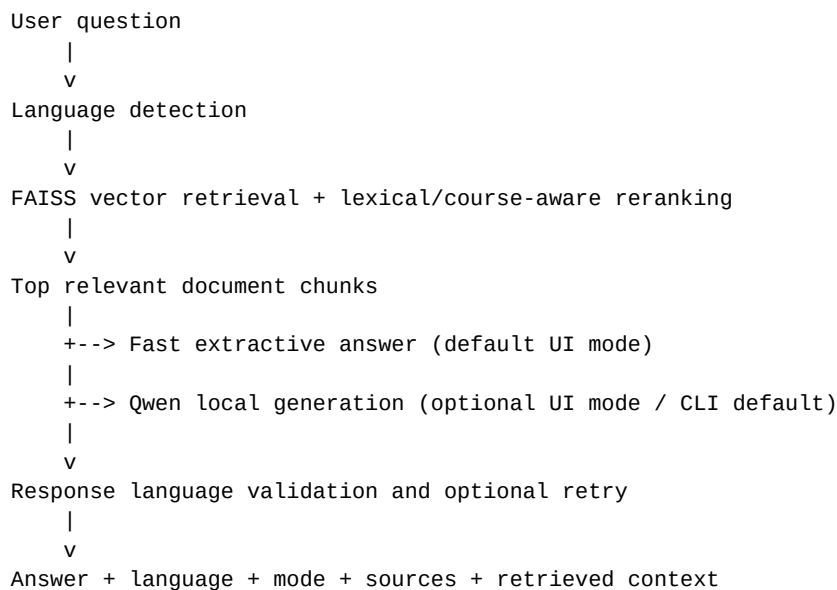
Knowledge base	Official university PDF documents
Languages	English, Bangla, Banglish
Default UI mode	Fast extractive answering
Optional mode	Local Qwen generation
Vector store	FAISS IndexFlatIP + metadata.pkl

1. Purpose

University Information Chatbot is a thesis prototype that answers questions about university courses and policies using only official PDF documents stored in data/documents/. It supports English, Bangla, and Banglish questions and returns an answer together with the retrieved source pages.

The project implements retrieval-augmented generation (RAG): the question is used to find relevant document text, and the answer is produced from that text instead of from an unrestricted knowledge source.

2. System Architecture



3. Knowledge Sources

The PDFs in data/documents/ are the knowledge base. The current documents are:

- document1.pdf: MTH 101, Math-I: Calculus I.
- document2.pdf: CSE 101, Computer Fundamentals and Programming.
- document3.pdf: ENG (CSE) 101, English.

The evaluation CSV in data/questions/questions.csv contains questions and expected answers for testing. It is not used as retrieval context and is not supplied to the answer generator.

4. Index-Building Process

Run python scripts/build_index.py after adding or changing PDFs.

1. pdf_loader.py reads every supported PDF with pypdf. 2. Text is extracted page by page and whitespace is normalized. 3. chunker.py splits each page into word windows of 700 words with 120 words of overlap. Chunks do not cross page boundaries. 4. embeddings.py encodes each chunk with BAAI/bge-m3 and normalized embeddings. 5. vector_store.py builds a FAISS IndexFlatIP inner-product index. 6. The index is written to vector_db/index.faiss. 7. The chunk records are written to vector_db/metadata.pkl.

Each metadata record contains text, source, page, chunk_id, word_start, and word_end. FAISS vector position and metadata list position correspond directly.

The current index contains 23 chunks with 1024-dimensional vectors.

5. Query Process, Step by Step

src/pipeline.py owns the end-to-end request flow.

1. Reject an empty question. 2. Detect the language with language_detector.py. 3. Load the FAISS index and metadata. 4. Select the answer mode. 5. Retrieve up to 15 candidates, then rerank and return the configured top K, normally 3. 6. Build the answer from the retrieved context. 7. Validate that the response language matches the question language. 8. If local generation used the wrong language, regenerate with a stricter language prompt. 9. Return the answer, detected language, response language, mode, sources, scores, pages, and debugging context.

6. Retrieval Behavior

retriever.py combines semantic and rule-based signals.

- BGE-M3 produces the semantic query vector.
- FAISS returns the nearest chunks by inner-product similarity.
- Question keywords add a small score boost when they appear in text or source names.
- Course hints such as CSE 101, MTH 101, or ENG (CSE) 101 boost the matching course.
- Course-specific requests preserve other chunks from the same course document, even when a later chunk does not repeat the course code.
- CLO questions promote chunks containing the requested CLO.
- Objective questions promote chunks containing Course Objectives.
- Unrelated course chunks are filtered when a specific course match is known.

fast_answer.py also provides lexical retrieval for fast mode. It scores token presence and exact course-code matches without loading the embedding model.

7. Answer Modes

Fast extractive mode

This is the default Streamlit mode. It is deterministic and fast. It reads the retrieved document text and extracts the requested fact when the question has a recognized structure.

Supported focused extraction includes:

- Course code and title, including ENG (CSE) 101.
- Course type and credit value.
- CLO 1 through CLO 4.
- Course objectives.
- Final Exam and Mid Term percentages.

For other questions, it selects relevant sentences from the retrieved chunks and prefixes them with a source-information label.

Local Qwen generation

When enabled, generator.py uses Qwen/Qwen2.5-1.5B-Instruct through Hugging Face Transformers. Sampling is disabled, so generation is deterministic for the same model and prompt. The prompt states that the model must use only supplied university context, avoid mixing courses, and obey the detected language.

The generator is cached after initialization. The first generated answer may be slow because the model must load and may need to download model files.

8. Language Handling

language_detector.py supports three labels:

- bangla: Bengali Unicode is present.
- banglish: Latin-script Bangla is detected from marker words or suffix patterns.
- english: Latin text without sufficient Banglish evidence.

Bengali script takes precedence. language_validator.py checks the output script/style and provides localized unsupported-answer messages. Bangla answers should use Bengali Unicode; Banglish answers should remain Latin-character Banglish; English answers should remain English.

9. User Interface

app.py provides the Streamlit interface.

- Shows whether the vector index is ready.
- Reports backend import problems.
- Accepts one question in a text field.
- Lets the user enable local Qwen generation.
- Displays elapsed pipeline status.
- Shows detected language, mode, answer, source file, page, score, and expandable retrieved chunks.

run_app.py launches Streamlit through the selected Python interpreter and defaults to port 8511. A custom port can be supplied, for example python run_app.py 8501.

10. Command Reference

Install dependencies:

```
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
```

Inspect extraction and chunking:

```
.\.venv\Scripts\python.exe scripts/ingest.py
```

Build or reuse the vector index:

```
.\.venv\Scripts\python.exe scripts/build_index.py
```

Ask one question with the local Qwen path:

```
.\.venv\Scripts\python.exe scripts/test_query.py "What is CLO 1 of CSE 101?"
```

Ask one question with fast extraction:

```
.\.venv\Scripts\python.exe scripts/test_query.py --fast "What percentage of the total marks is allocated to CLO 1 of CSE 101?"
```

Run evaluation:

```
.\.venv\Scripts\python.exe scripts/evaluate.py
```

Run environment diagnostics:

```
.\.venv\Scripts\python.exe diagnose_runtime.py
```

Run the UI:

```
.\.venv\Scripts\python.exe run_app.py 8501
```

11. Evaluation Process

src/evaluator.py loads every available English, Bengali, and Banglish variant from the CSV, calls the pipeline, and writes results/evaluation_results.csv.

Each row records the question, reference answer, generated answer, expected and detected languages, response language, language consistency, retrieved sources, pages, and scores. Factual correctness, relevance, completeness, and groundedness currently require manual review.

The dataset has expected answer text but no labeled source or page for each question. Therefore Hit@K, Recall@K, and MRR cannot be calculated reliably until source annotations are added.

12. Runtime and Dependency Notes

The project uses a Python virtual environment at .venv. Core runtime packages are torch, torchvision, transformers, sentence-transformers, faiss-cpu, pypdf, and streamlit.

The current compatible CPU pairing is Torch 2.6 with torchvision 0.21. The torchvision dependency was added because Transformers may inspect optional vision modules during Streamlit startup; without it, startup produced ModuleNotFoundError: No module named 'torchvision'.

diagnose_runtime.py imports the main packages and prints versions and tracebacks. A Windows c10.dll initialization error indicates a PyTorch environment problem, not necessarily a problem in RAG logic.

13. Fixes Implemented During This Project Session

- Added targeted course-code and title extraction for metadata questions.
- Added CLO extraction so CLO questions do not fall back to course metadata.
- Added objective extraction for objective questions.
- Added Final Exam and Mid Term percentage extraction.
- Made metadata answers intent-specific: type and credit questions no longer return unrelated title details.
- Improved course-aware retrieval so later same-course chunks can be selected for CLO and objective questions.

- Added torchvision>=0.21.0,<0.22.0 to requirements.txt for the Torch 2.6 environment.

14. Known Limitations

- PDF extraction works for text PDFs; scanned image PDFs require OCR.
- Fixed word-based chunking can split tables and sentences awkwardly.
- The fast extractor is strongest for the recognized question patterns and is primarily English-oriented.
- Language detection is heuristic and can misclassify short mixed-language questions.
- Local generation requires model downloads, sufficient disk space, and compatible CPU/GPU runtime dependencies.
- The application currently runs embeddings on CPU.
- Retrieval scores are useful for ranking but are not probabilities.
- Evaluation still needs human factual and groundedness review plus source annotations for formal retrieval metrics.

15. Recommended Future Improvements

1. Add automated tests for every focused extractor and language variant. 2. Add source-document and page labels to the evaluation CSV. 3. Improve PDF table extraction and optionally add OCR for scanned documents. 4. Use a structured answer schema for exact fields such as grades, marks, weeks, and course outcomes. 5. Add confidence thresholds and a clearer unsupported-answer path when retrieval is weak. 6. Add a document refresh command that rebuilds the index after detecting changed PDFs. 7. Compare fast extraction and Qwen generation against manually reviewed evaluation results.

16. End-to-End Summary

PDF files are converted into page text, page text becomes overlapping chunks, chunks become normalized BGE-M3 vectors, and vectors plus metadata form the FAISS knowledge index. A user question is language-detected, semantically and lexically retrieved, course-aware reranked, and answered either by deterministic extraction or by a local Qwen model. The response is language-checked and returned with traceable source pages and retrieved text. Evaluation records the result for later manual quality review.